

GARDENIA

Ecosistema de Agentes de IA para Análisis Ómico
GUÍA DE DESARROLLO

GARDENIA

Ecosistema de Agentes de IA para Análisis Ómico
GUÍA DE DESARROLLO

PARTE 1 — VISIÓN GENERAL Y ARQUITECTURA EN CAPAS

1. Qué es Gardenia y cómo está organizado

Gardenia es una plataforma de análisis multi-ómico que automatiza el ciclo completo de un experimento biomédico — desde la lectura de un paper científico hasta el reporte clínico final — usando un ecosistema coordinado de agentes de IA especializados. El científico mantiene el control en cada decisión crítica mediante un sistema Human-in-the-Loop.

Principio central: ningún modelo hace todo. Gardenia usa una estrategia de 3 capas complementarias donde cada capa resuelve un problema diferente:

- Capa 1 — Orquestación: razonar, generar código, coordinar agentes
- Capa 2 — Dominio médico: entender papers, ontologías, KEGG, metadatos clínicos
- Capa 3 — Ómicas especializadas: embeddings celulares, batch effects, multi-ómica

1.1 Las 3 Capas de Modelos

Capa	Función principal	Modelos recomendados	Hardware mínimo
Capa 1 — Orquestación	Razonamiento, código Python/R, coordinación	Qwen2.5-Coder-32B, DeepSeek-R1-32B, Llama 3.3 70B	1× A100 80GB
Capa 2 — Dominio Médico	Papers, KEGG, GO, metadatos clínicos, SNOMED	MedGemma 27B, OpenBioLLM-70B, BioMistral-7B	1× A100 80GB
Capa 3 — Ómicas	Embeddings celulares, batch effects, multi-ómica	scGPT, Geneformer, scBERT, MedSigLIP 400M	1× RTX 4090 (algunos)

1.2 Las 2 Fases de Implementación

Fase	Agentes incluidos	Objetivo
FASE 1 — MVP	Diseñador Clínico, Guardián de Código, Analista Comparativo, Narrador Biomédico	Ciclo completo de un experimento: diseño → ejecución → reporte
FASE 2 — Roadmap	Curador de Metadatos, Integrador Multi-Ómico, Operador de Referencias Genómicas	Flujos avanzados: armonización clínica, multi-ómica, gestión de referencias

1.3 Stack Tecnológico General

Componente	Herramienta elegida	Por qué
Serving de LLMs	vLLM con OpenAI-compatible API	Soporta multi-LoRA nativo: un modelo base, N adaptadores
Fine-tuning	Unsloth + QLoRA (TRL)	El más eficiente disponible: reduce VRAM un 70%
Modelos ómicos	Microservicios REST independientes	scGPT y Geneformer no son conversacionales
RAG biomédico	ChromaDB o FAISS + BioMistral-7B	Búsqueda local sobre PubMed sin internet en producción
IPC Python/R	Apache Arrow + unified_bridge	Serialización cero-copia entre lenguajes
Frontend	React + React Flow	Visualización y edición de grafos de workflow

2. Selección de Modelos por Capa

2.1 Capa 1 — Orquestación y Código

Modelo	Tamaño	Licencia	Para qué se usa en Gardenia	Por qué este modelo
Qwen2.5-Coder-32B-Instruct	32B	Apache 2.0	Code Guardian completo, generación de JSON ReactFlow, análisis de stack traces IPC	Estado del arte en generación de código. Maneja Python, R y JSON estructurado mejor que cualquier otro open-source.
DeepSeek-R1-Distill-Qwen-32B	32B	MIT	Diseñador Clínico: decidir entre metodologías (DESeq2 vs edgeR, STAR vs HISAT2)	Chain-of-thought explícita. Muestra su razonamiento paso a paso, ideal para Human-in-the-Loop.
Llama 3.3 70B Instruct	70B	Meta Llama	Narrador Biomédico, Curador de Metadatos, conversación con el usuario	El más equilibrado para narrativa científica fluida. Mejor soporte de ecosistema (Ollama, vLLM).
Qwen2.5-Coder-7B	7B	Apache 2.0	Local DB Manager (tareas DevOps ligeras)	Suficiente para comandos de shell y gestión de archivos. Cabe en GPU pequeña.

2.2 Capa 2 — Dominio Médico y Biomédico

Modelo	Datos de entrenamiento clave	Para qué en Gardenia	Por qué
MedGemma 27B (Google)	TCGA (genómica cáncer), MIMIC, PubMed, histopatología, imágenes médicas	Parse de papers, Query KEGG/GO, interpretar plots con visión, contexto oncológico	Fue entrenado en TCGA: conoce patrones de expresión génica en cáncer. Multimodal: entiende imágenes y texto.
OpenBioLLM-70B (Saama)	Fine-tune extenso sobre Llama-3 con datos biomédicos	Map_To_Standard_Vocabulary: SNOMED, OMOP, HL7. Terminología clínica precisa	Supera a GPT-4 en benchmarks biomédicos específicos. Mejor para nomenclatura clínica estandarizada.
BioMistral-7B (EPFL)	PubMed Central completo (pre-entrenamiento continuo desde Mistral)	RAG en tiempo real sobre literatura para el Diseñador Clínico	Ligero (7B), ideal para deploy local sin GPU masiva. Especializado en texto biomédico.

2.3 Capa 3 — Foundation Models Ómicos

Estos NO son LLMs conversacionales. Son transformers entrenados directamente en datos biológicos crudos. Se despliegan como microservicios REST independientes que los agentes llaman internamente.

Modelo	Entrenado en	Capacidades	Agente que lo usa
scGPT ★	33M+ perfiles de células humanas (scRNA-seq)	Corrección de batch effects, integración multi-ómica scRNA+scATAC, anotación de tipos celulares, predicción de perturbaciones	Analista Comparativo, Integrador Multi-Ómico, Narrador Biomédico
Geneformer (Broadinstitute)	30M perfiles celulares humanos	Predicción de redes reguladoras génicas, priorización de genes objetivo para terapia génica	Narrador Biomédico, Diseñador Clínico (validación in-silico)
scBERT (Tencent AI)	Datos scRNA-seq de múltiples tejidos	Anotación de tipos celulares. Solo 85M params, muy ligero	Narrador Biomédico (clasificación de clusters UMAP)
MedSigLIP 400M (Google)	Imágenes médicas diversas (adaptado de SigLIP)	Encoding de imágenes biomédicas: Volcano Plots, PCAs, Heatmaps, histología	Narrador Biomédico — Analyze_Visual_Output

PARTE 3 — LOS 7 AGENTES: PASOS Y RESPONSABILIDADES

3. Especificación de Agentes

Cada agente tiene una responsabilidad única y no solapada. El flujo de datos va: Curador → Diseñador → Guardián → (ejecución) → Analista → Narrador. Los agentes de Fase 2 enriquecen este ciclo para casos de mayor complejidad.

— FASE 1: NÚCLEO MVP —

Agente 1 — Diseñador Clínico (Clinical Designer) [FASE 1 — MVP]	
Misión	Leer un paper o descripción textual y convertirlo en un workflow válido de React Flow. Pausa en decisiones metodológicas clave para que el usuario apruebe.
Herramientas	► Parse_PDF_Document — extrae texto, genes, metodología de un paper ► Search_Biomedical_DB — RAG sobre PubMed/NCBI para llenar vacíos metodológicos ► Ask_User_Decision — pausa el flujo y envía opciones a la UI (Human-in-the-Loop) ► Search_Available_Nodes — busca en la DB local de Gardenia qué nodos/herramientas existen ► Recommend_Plugin_Installation — si falta un nodo, sugiere instalar plugin del Marketplace ► Create_ReactFlow_Graph — ensambla nodos y conexiones en el lienzo del frontend
Modelos	→ DeepSeek-R1-32B → razonamiento metodológico explícito (¿DESeq2 o edgeR?) → MedGemma 27B → comprensión profunda del paper biomédico → BioMistral-7B → RAG sobre PubMed para validar decisiones → Modelo fine-tuneado con Galaxy IWC → patrones de workflow ómico real
Por qué	<i>El Diseñador necesita razonar en voz alta (DeepSeek-R1), entender biología profunda (MedGemma) y conocer workflows reales validados (fine-tuning con IWC). Ningún modelo solo cumple los tres roles.</i>

Agente 2 — Guardián de Código (Code Guardian) [FASE 1 — MVP]	
Misión	Vigilar que el puente IPC Python ↔ R no colapse. Actúa ANTES de ejecutar (pre-vuelo) y DURANTE (rescate de errores en tiempo real).
Herramientas	► Read_Node_Code / Write_Node_Code — inspecciona y modifica scripts de nodos ► Static_Type_Check_IPC — simula el flujo de datos entre lenguajes antes de ejecutar ► Inspect_VariableRegistry — monitorea RAM para alertar sobre Out-Of-Memory ► Analyze_IPC_StackTrace — si un nodo falla, recolecta logs y genera hotfix automático
Modelos	→ Qwen2.5-Coder-32B → especialista en código: análisis estático, tipos Python/R, Arrow → Adaptador LoRA fine-tuneado → patrones comunes de error en IPC biomédico
Por qué	<i>El Guardián es puramente un problema de código, no de biología. Qwen2.5-Coder es el mejor modelo open-source para este trabajo. No necesita conocimiento médico.</i>

Agente 3 — Analista Comparativo (Comparative Analyst) [FASE 1 — MVP]	
Misión	Diagnosticar y explicar divergencias entre dos ramas del workflow (Control vs. Tratamiento) o entre dos ejecuciones con parámetros diferentes.
Herramientas	► Compare_Distributions — contraste estadístico entre salidas de dos ramas ► Detect_Batch_Effect — detecta varianza técnica no deseada en matrices ómicas ► Diff_Node_Parameters — explica cómo un cambio de parámetro (ej. k en k-means) afectó el resultado final
Modelos	→ Qwen2.5-Coder-32B → genera el análisis estadístico y el código comparativo → BioMistral-7B → contextualiza biológicamente las diferencias encontradas → scGPT → corrección y análisis de batch effects en datos de single-cell
Por qué	<i>La mezcla de un modelo de código (Qwen) para el análisis técnico y uno biomédico (BioMistral) para la interpretación biológica es más poderosa que usar uno solo para ambas tareas.</i>

Agente 4 — Narrador Biomédico (Biomedical Storyteller) [FASE 1 — MVP]	
Misión	Convertir resultados técnicos (DataFrames, plots) en hallazgos biológicos comprensibles para publicación o revisión clínica.
Herramientas	► Summarize_Dataframe — extrae estadísticas clave sin pasar matrices gigantes al LLM ► Analyze_Visual_Output — interpreta plots (Volcano, PCA, Heatmap) con visión artificial ► Query_KEGG_GO — enriquecimiento automático de vías metabólicas sobre lista de genes ► Export_Markdown_Report — compila el reporte final en PDF o Markdown
Modelos	→ Llama 3.3 70B → narrativa científica fluida y comprensible → MedGemma 27B Multimodal → interpreta imágenes biomédicas (visión + texto) → MedSigLIP 400M → encoding especializado de plots e imágenes médicas → OpenBioLLM-70B → terminología clínica precisa en el reporte final
Por qué	<i>El Narrador necesita visión para interpretar plots (MedGemma + MedSigLIP), conocimiento clínico para el reporte (OpenBioLLM) y fluidez narrativa para el texto final (Llama 3.3).</i>

— FASE 2: ESCALABILIDAD MULTI-ÓMICA —

Agente 5 — Curador de Metadatos (Metadata Curator) [FASE 2 — Roadmap]	
Misión	Armonizar y estandarizar matrices clínicas antes de inyectarlas al motor de análisis. Actúa ANTES del Diseñador Clínico.
Herramientas	► Analyze_Tabular_Header — escanea columnas buscando inconsistencias de nomenclatura ► Map_To_Standard_Vocabulary — estandariza con SNOMED / OMOP / HL7

	► Impute_Missing_Clinical_Data — sugiere y aplica estrategias de imputación
Modelos	→ Llama 3.3 70B → comprensión de tablas clínicas heterogéneas → OpenBioLLM-70B → mapeo preciso a vocabularios médicos estandarizados
Por qué	OpenBioLLM fue entrenado específicamente en terminología clínica estructurada, lo que lo hace superior para el mapeo SNOMED/OMOP frente a modelos generales.

Agente 6 — Integrador Multi-Ómico (Multi-Omics Synthesizer) [FASE 2 — Roadmap]	
Misión	Alinear matrices de distintas modalidades (ej. Transcriptómica + Proteómica) con dimensiones incompatibles para análisis conjunto.
Herramientas	► Identify_Matrix_Overlap — encuentra intersección de IDs entre matrices heterogéneas ► Suggest_Integration_Method — recomienda MOFA, SNF o Seurat WNN según la combinación ► Build_Dimensionality_Bridge — construye el código puente matemático entre dimensiones
Modelos	→ Qwen2.5-Coder-32B → código puente matemático entre matrices → MedGemma 27B → contexto sobre las modalidades ómicas combinadas → scGPT multi-omics → integración nativa scRNA + scATAC en espacio latente unificado
Por qué	scGPT fue pre-entrenado explícitamente en integración multi-ómica — no hay que 'convencer' al modelo de cómo alinear modalidades, ya lo sabe por entrenamiento.

Agente 7 — Operador de Referencias Genómicas (Local DB Manager) [FASE 2 — Roadmap]	
Misión	Gestionar la descarga, indexación y caché de archivos de referencia masivos (FASTA, GTF, índices STAR/HISAT2) sin interrumpir el workflow del usuario.
Herramientas	► Check_Local_Reference_Exists — verifica si el índice genómico ya existe en caché ► Fetch_Ensembl_Reference — descarga secuencias primarias de Ensembl/UCSC ► Spawn_Background_Indexer — lanza la compilación del índice en segundo plano con % de progreso
Modelos	→ Qwen2.5-Coder-7B → suficiente para operaciones de sistema y comandos de shell
Por qué	Este agente no necesita entender biología, solo ejecutar comandos del sistema correctamente. El modelo más pequeño (7B) es suficiente y más eficiente en recursos.

4. Recolección de Datos Offline desde Galaxy

Galaxy IWC y su ecosistema ofrecen cientos de workflows biomédicos curados y revisados por pares. Se usan exclusivamente como corpus estático de entrenamiento. En producción, Gardenia no depende de Galaxy ni se conecta a ninguna instancia externa.

- ✓

Todo el proceso es offline: se descarga una sola vez con git clone o curl.
- ✓

No se necesita API key ni autenticación para ninguna de las fuentes.
- ✓

Los README.md del IWC ya contienen explicaciones escritas por bioinformáticos humanos — no necesitas un modelo para generarlas.

4.1 Las 3 Fuentes de Datos

Fuente	Cómo obtenerla	Workflows disponibles	Calidad de metadata
IWC GitHub (galaxyproject/iwc)	git clone github.com/galaxyproject/iwc	~180 workflows	Excelente — 100% con README.md
WorkflowHub.eu	API pública REST, sin auth, filtrar por Galaxy	~1,500 workflows	Buena — ~800 con descripción completa
Galaxy Training Network (GTN)	git clone github.com/galaxyproject/training-material	~230 tutoriales con workflows embebidos	Excelente — contexto biológico extenso

4.2 Estructura de un Workflow IWC

Cada workflow en el IWC viene en una carpeta con tres archivos. Esta estructura ya forma el par natural instrucción → respuesta que necesita el fine-tuning:

Archivo	Contenido	Rol en el fine-tuning
workflow.ga	JSON con los pasos, herramientas y conexiones del workflow	La 'respuesta' — lo que el modelo aprende a generar
README.md	Descripción biológica del workflow, escrita por bioinformáticos	El 'prompt' — la instrucción en lenguaje natural
.dockstore.yml	Metadata: autores, versiones, tags de categoría	Enriquece el contexto del ejemplo de entrenamiento

4.3 Categorías Ómicas Disponibles

Categoría	Workflows típicos	Relevancia para Gardenia
Transcriptómica	RNA-seq bulk, Single-cell 10X, DESeq2, edgeR	Alta — uso directo para el Diseñador Clínico
Epigenómica	ChIP-seq, ATAC-seq, Hi-C, bisulfite-seq	Alta — diseño de experimentos epigenéticos
Variante / WGS	GATK, Mutect2, variant annotation, somatic variants	Alta — oncología y medicina de precisión
Proteómica	DIA-NN, MaxQuant, TMT quantification	Media — Integrador Multi-Ómico
Metagenómica	16S rRNA, shotgun metagenomics, QIIME2	Media — microbioma
Ensamblado	De novo genome assembly, annotation	Media — Local DB Manager

5. Del Corpus Galaxy al Dataset de Entrenamiento

5.1 Tipos de Pares de Entrenamiento

Cada workflow Galaxy genera múltiples ejemplos de entrenamiento, uno diseñado para cada agente. Un solo workflow bien documentado puede producir 3 a 4 ejemplos de alta calidad.

Tipo de ejemplo	Prompt (entrada)	Respuesta esperada (salida)	Agente
Descripción → pasos	Texto del README.md: 'Workflow para RNA-seq de células T...'	Lista ordenada de herramientas con conexiones	Diseñador Clínico
Descripción → JSON ReactFlow	Misma descripción del README	JSON con nodes[] y edges[] del grafo de Gardenia	Diseñador Clínico
Workflow → explicación biológica	Nombre y herramientas del workflow	Párrafo explicando qué hace biológicamente y para qué sirve	Narrador Biomédico
Steps → código Python/R con Arrow	Herramientas consecutivas del workflow	Código de conversión seguro con Apache Arrow entre nodos	Code Guardian
Metadata clínica → SNOMED	Columnas de un CSV clínico con nombres heterogéneos	Mapeo estándar a vocabulario SNOMED/OMOP	Curador de Metadatos

5.2 Volumen Final Estimado del Dataset

Fuente	Workflows útiles	Ejemplos generados	Agente principal
IWC GitHub	~150	~450–600 pares	Diseñador, Narrador, Guardián
WorkflowHub ómicas	~600	~1,800–2,400 pares	Diseñador, Narrador
GTN tutoriales	~200	~600–800 pares (mayor contexto biológico)	Narrador, Diseñador
TOTAL	~950	~3,000–4,000 pares de alta calidad	Todos los agentes

💡 ¿Por qué son suficientes ~3,000 ejemplos?
El fine-tuning de dominio específico no necesita millones de ejemplos como el pre-entrenamiento general.
Con datos de alta calidad y muy especializados, 1,000–5,000 ejemplos son suficientes para adaptar el comportamiento de un modelo.
El proyecto Stanford Alpaca instruyó modelos completos con 52K ejemplos sintéticos. Con ómicas, el dominio es más estrecho → menos datos, mejores resultados.

5.3 Proceso de Fine-Tuning Paso a Paso

- Descargar el corpus: git clone de IWC, GTN y scraping de WorkflowHub con la API pública.
- Parsear los archivos .ga: extraer name, description, steps, tool_ids y conexiones.
- Leer los README.md correspondientes como descripciones en lenguaje natural.
- Generar pares instrucción-respuesta: un script convierte cada workflow en 3-4 ejemplos de entrenamiento en formato JSONL.
- Revisión de calidad manual del 10%: verificar que los ejemplos son biológicamente correctos y las respuestas tienen sentido.
- Fine-tuning con QLoRA usando Unsloth: entrenar adaptadores LoRA separados por agente sobre el modelo base correspondiente.
- Evaluación: verificar que el modelo genera JSON ReactFlow válido, que los pasos biológicos son correctos y que el código IPC no tiene errores de tipos.
- Despliegue: servir el modelo base UNA SOLA VEZ con vLLM y cargar los adaptadores por agente según la solicitud.

5.4 Configuración Recomendada de Fine-Tuning

Parámetro	Valor recomendado	Razón
Método	QLoRA (Quantized LoRA)	Reduce VRAM de 70GB a ~20GB manteniendo calidad comparable
LoRA rank (r)	64	Rank más alto para tareas estructuradas como JSON — más capacidad de adaptación
LoRA alpha	128 (= 2× rank)	Convención estándar: alpha = 2× rank para escala estable
Epochs	3	Suficiente con 3,000–5,000 ejemplos para evitar overfitting
Learning rate	2e-4	Estándar para QLoRA con scheduler cosine
Batch efectivo	16 (batch=2, grad_accum=8)	Simula batch grande ahorrando VRAM
Sequence length	4,096 tokens	Los workflows Galaxy pueden ser verbosos en JSON
Librería	Unsloth + TRL (SFTTrainer)	La más eficiente: 2× más rápido que HuggingFace puro

PARTE 6 — INFRAESTRUCTURA Y DESPLIEGUE

6. Arquitectura de Despliegue de Gardenia

6.1 Estrategia de Serving: Un Modelo Base, N Adaptadores

La clave de la eficiencia de Gardenia es que el modelo base (ej. Qwen2.5-Coder-32B) se carga UNA SOLA VEZ en GPU. Los adaptadores LoRA de cada agente (~150-300MB cada uno) se intercambian en microsegundos sin recargar el modelo, gracias al soporte multi-LoRA nativo de vLLM.

Componente	Herramienta	Rol
Serving de LLMs (Capa 1 y 2)	vLLM con multi-LoRA	Sirve todos los agentes desde un solo proceso, seleccionando adaptador por request
Modelos ómicos (Capa 3)	FastAPI / Flask microservicios	scGPT, Geneformer y scBERT como APIs REST independientes por capacidad
RAG biomédico	ChromaDB + BioMistral-7B	Índice vectorial local de PubMed — sin internet en producción
Orquestación de agentes	LangGraph o CrewAI	Define el flujo entre agentes, el state machine y el Human-in-the-Loop
IPC Python/R	Apache Arrow + unified_bridge	Serialización cero-copia para pasar matrices entre lenguajes sin copiar datos
Frontend	React + React Flow	Lienzo visual del workflow — los agentes escriben en él vía API

6.2 Flujo de Datos entre Agentes

Pa so	Quién actúa	Qué hace	Qué pasa si hay error
1	Curador de Metadatos	Valida y estandariza los datos clínicos de entrada (CSV, tablas)	Alerta al usuario con sugerencias de corrección
2	Diseñador Clínico	Lee el paper/descripción, consulta PubMed, propone workflow, pide aprobación al usuario	Vuelve a Ask_User_Decision con opciones alternativas
3	Guardián de Código	Revisa tipos IPC, simula el flujo de datos, alerta sobre OOM antes de ejecutar	Sugiere conversiones Arrow o pre-procesado para reducir memoria
4	(Motor de ejecución)	Ejecuta los nodos Python/R en orden según el grafo	Guardián captura el stack trace y genera hotfix automático
5	Analista Comparativo	Compara resultados entre ramas del workflow o con ejecuciones anteriores	Muestra divergencia y posibles causas (batch effect, parámetro diferente)
6	Narrador Biomédico	Interpreta los resultados finales y genera el reporte clínico	Solicita más contexto al usuario si los resultados son ambiguos

6.3 Human-in-the-Loop: Cuándo Parar y Preguntar

Agente	Cuándo para	Qué le pregunta al usuario
Diseñador Clínico	Cuando hay 2+ metodologías igualmente válidas	¿Prefieres DESeq2 o edgeR para este análisis diferencial?
Diseñador Clínico	Cuando el paper usa un nodo que no existe en Gardenia	¿Instalo el plugin X del Marketplace para continuar?
Guardián de Código	Cuando detecta riesgo alto de OOM (>80% RAM)	El paso Y usará ~45GB. ¿Pre-proceso los datos o aumentas la RAM?
Analista Comparativo	Cuando detecta un batch effect significativo	¿El batch effect entre muestras refleja biología real o es ruido técnico?
Narrador Biomédico	Cuando un resultado es inesperado o ambiguo	Los genes del cluster 3 no tienen ontología clara. ¿Tienes contexto adicional?

6.4 Hardware Mínimo Recomendado

Configuración	GPU	RAM	Qué puedes correr
Desarrollo / pruebas	1× RTX 4090 (24GB)	64GB RAM sistema	Capa 3 completa + BioMistral-7B + Qwen2.5-Coder-7B
MVP producción	1× A100 80GB	128GB RAM sistema	Qwen2.5-Coder-32B + MedGemma 27B + todos los adaptadores LoRA
Producción completa	2× A100 80GB	256GB RAM sistema	Llama 3.3 70B + OpenBioLLM-70B + toda la Capa 3
Alternativa cloud	AWS p4d.24xlarge o GCP a2-ultragpu	—	Toda la arquitectura completa de Gardenia

PARTE 7 — ROADMAP DE DESARROLLO

7. Plan de Implementación por Fases

Fase 1 — MVP

Semana	Objetivo	Entregable
1–2	Descargar y parsear corpus Galaxy IWC + GTN. Escribir el script de conversión a pares JSONL.	Dataset inicial: ~1,500 ejemplos de alta calidad
2–3	Revisión manual de calidad del 10% del dataset. Limpieza y filtrado.	Dataset validado listo para entrenamiento
3–4	Fine-tuning QLoRA del Diseñador Clínico sobre Qwen2.5-Coder-32B con Unsloth.	Adaptador LoRA del Diseñador: ~150MB
4–5	Fine-tuning del Guardián de Código y Narrador Biomédico. Evaluación de los 3 adaptadores.	3 adaptadores LoRA funcionales
5–6	Despliegue vLLM con multi-LoRA. Integrar MedGemma 27B y MedSigLIP como servicios.	Stack de serving completo
6–7	Conectar agentes al frontend React Flow. Implementar Human-in-the-Loop (Ask_User_Decision).	MVP funcional end-to-end
7–8	Pruebas con workflows reales de usuarios. Recolectar feedback y refinar.	MVP validado en producción

Fase 2 — Escalabilidad

Semana	Objetivo	Entregable
9–10	Harvesting de WorkflowHub (~600 workflows ómicos). Ampliar dataset con pares de multi-ómica.	Dataset ampliado: ~3,500–4,000 ejemplos totales
10–11	Desarrollar el Curador de Metadatos con OpenBioLLM-70B + SNOMED.	Agente 5 funcional
11–12	Desarrollar Integrador Multi-Ómico con scGPT como microservicio.	Agente 6 + microservicio scGPT
12–13	Desarrollar Local DB Manager (Agente 7) para gestión de referencias Ensembl/UCSC.	Agente 7 funcional
13–14	Orquestación completa con LangGraph: state machine de los 7 agentes.	Pipeline completo multi-agente
14–16	Optimización, monitoreo, logging de errores IPC. Preparar para usuarios reales.	Sistema estable en producción

 Flywheel de datos: cada workflow creado por un usuario real en Gardenia (con su consentimiento) se convierte en dato de entrenamiento adicional. Con el tiempo, el sistema se vuelve progresivamente más inteligente en el dominio específico de tus propios usuarios — un ciclo virtuoso de mejora continua.

RESUMEN EJECUTIVO

Decisión	Elección en Gardenia	Razón clave
Modelo estrella para código	Qwen2.5-Coder-32B-Instruct	Estado del arte open-source en generación de código y JSON estructurado
Modelo estrella biomédico	MedGemma 27B (Google)	Entrenado en TCGA genómica + multimodal (texto + imágenes médicas)
Modelo de razonamiento	DeepSeek-R1-Distill-Qwen-32B	Chain-of-thought explícito, ideal para decisiones metodológicas visibles
Foundation model ómico	scGPT (33M células)	Cubre batch effects, multi-ómica y anotación celular en un solo modelo
Datos de fine-tuning	Galaxy IWC + GTN (offline)	Workflows curados por bioinformáticos reales, con descripción ya incluida
Método de fine-tuning	QLoRA con Unsloth	70% menos VRAM, 2× más rápido, adaptadores de solo ~150MB por agente
Serving en producción	vLLM con multi-LoRA	Un modelo base, N adaptadores intercambiables sin recargar la GPU
IPC Python/R	Apache Arrow + unified_bridge	Cero-copia, tipado fuerte entre lenguajes, previene el 80% de los crashes